

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Frameworks.

Instalación del Java Development Kit (JDK)

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Angel Frausto Robles

id634296@gmail.com

7CM2

11 de septiembre de 2026

Índice

1. Introducción	2
2. Desarrollo	2
2.1. Descarga del JDK	2
2.2. Instalación	2
2.3. Configuración de la variable JAVA_HOME	3
2.4. Verificación	3
3. Conclusión	4
4. Referencias Bibliográficas	5

Índice de figuras

1.	Página de descarga del JDK	2
2.	JDK registrado en Windows tras la instalación (Configuración → Aplicaciones)	2
3.	Variable de entorno <code>JAVA_HOME</code> configurada en Windows	3
4.	Verificación de <code>java -version</code> y <code>JAVA_HOME</code> en la terminal	4

1. Introducción

El Java Development Kit (JDK) es el paquete de herramientas que permite escribir, compilar y ejecutar programas en el lenguaje Java. Incluye, entre otras piezas, el compilador (`javac`), la máquina virtual de Java (`java`) y las bibliotecas estándar del lenguaje. Sin un JDK instalado, un proyecto Java no puede ni siquiera compilarse, y esa es la razón por la que su instalación es el primer paso obligatorio antes de trabajar con cualquier framework construido sobre Java, entre ellos Spring, que sostiene la parte de backend de este curso.

Java como especificación es propiedad de Oracle Corporation [1], que además distribuye su propia implementación del JDK. Sin embargo, existen otras distribuciones que implementan la misma especificación y pasan el mismo conjunto de pruebas de compatibilidad (TCK), entre ellas Eclipse Temurin, mantenida por la fundación Eclipse Adoptium [2]. Es la distribución que se documenta en este reporte.

El objetivo de este reporte es doble: identificar cuál es la variable de entorno que Java necesita para operar correctamente (`JAVA_HOME`) y documentar el proceso completo de descarga, instalación y configuración de esa variable en el sistema operativo, dejando evidencia de cada paso y de la verificación final.

2. Desarrollo

2.1. Descarga del JDK

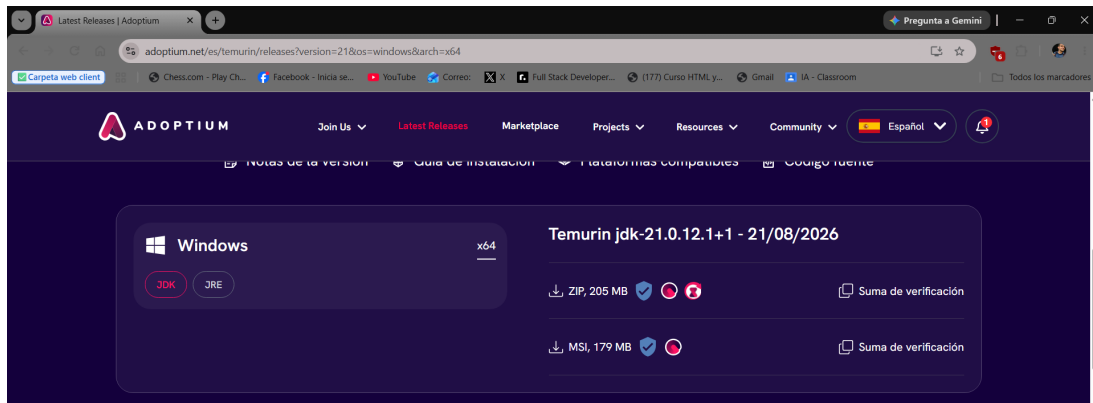


Figura 1: Página de descarga del JDK

El JDK se descargó del sitio oficial de la distribución elegida [2], seleccionando la versión de soporte extendido (LTS) más reciente disponible para el sistema operativo de la máquina de trabajo (Windows, arquitectura x64). Optar por una versión LTS, en vez de una versión intermedia de ciclo corto, garantiza soporte y parches de seguridad durante varios años. Es un criterio relevante porque el proyecto del semestre se va a sostener sobre esta misma instalación.

2.2. Instalación

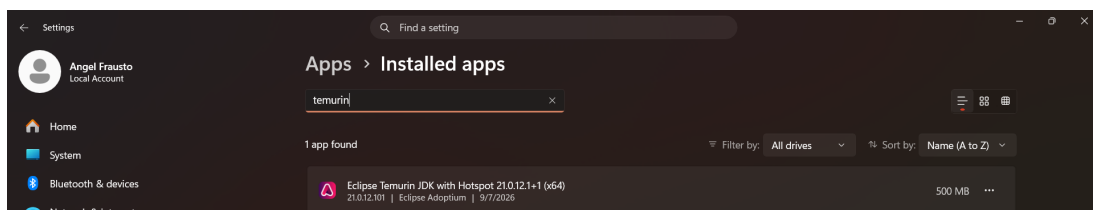


Figura 2: JDK registrado en Windows tras la instalación (Configuración → Aplicaciones)

La instalación se realizó con el instalador `.msi` para Windows, que ejecuta el proceso completo de forma asistida: copia los binarios a `Program Files`, registra el JDK en el sistema y ofrece la opción de agregarlo automáticamente a la variable `PATH`. Aun con esa opción activada, es necesario revisar (y, en caso de que falte, declararla a mano) la variable `JAVA_HOME`, porque muchas herramientas de compilación (Maven, Gradle) y varios IDEs la consultan directamente en vez de inferirla a partir del `PATH`. La instalación queda registrada en el sistema y puede confirmarse desde *Configuración* → *Aplicaciones*, donde Windows lista a Eclipse Temurin JDK como programa instalado.

2.3. Configuración de la variable JAVA_HOME

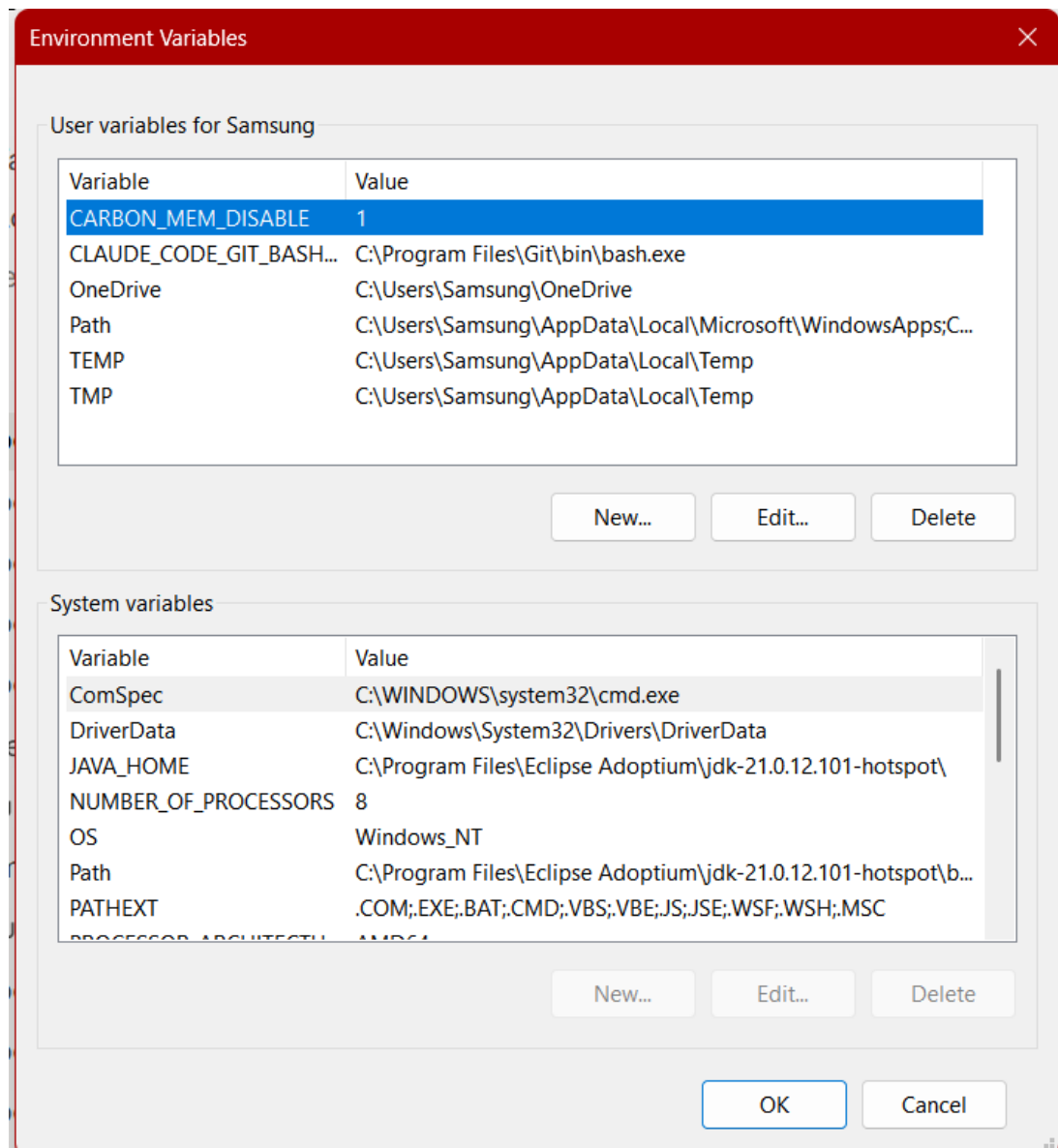


Figura 3: Variable de entorno JAVA_HOME configurada en Windows

En Windows, las variables de entorno se configuran desde *Panel de Control* → *Sistema* → *Configuración avanzada del sistema* → *Variables de entorno* [3]. Ahí se creó una variable nueva de usuario llamada `JAVA_HOME`, cuyo valor apunta a la carpeta raíz de la instalación del JDK (no a la subcarpeta `bin`, que es el error más común en este paso), y se agregó `%JAVA_HOME%\bin` al final de la variable `Path`, para que el sistema encuentre los ejecutables `java` y `javac` desde cualquier terminal.

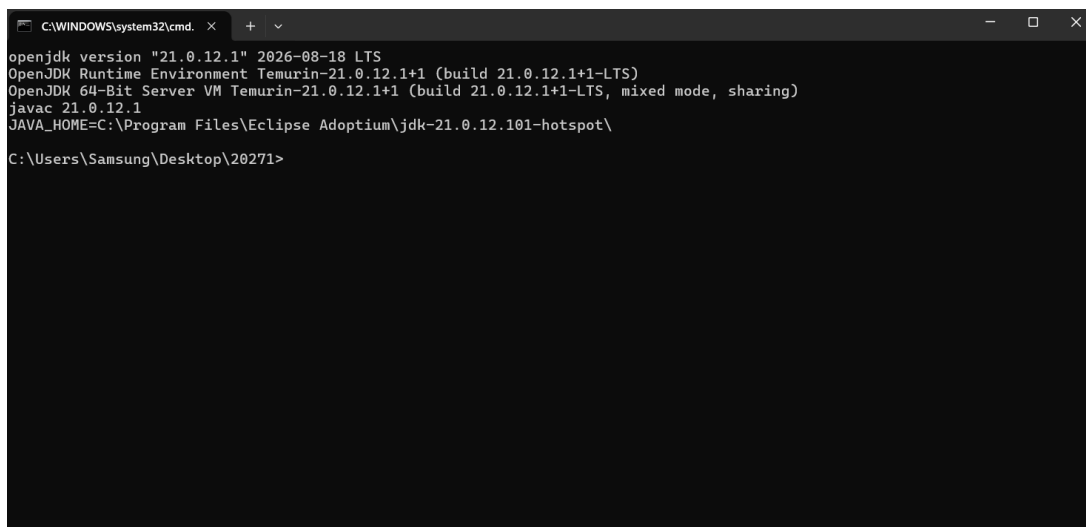
2.4. Verificación

Con la variable ya configurada, se abrió una terminal nueva (paso obligatorio: una terminal ya abierta conserva el entorno con el que arrancó y no ve variables agregadas después) y se confirmó la instalación con los comandos `java -version` y `javac -version`:

Código 1: Verificación del JDK instalado y de la variable `JAVA_HOME`

```
1 $ java -version
2 openjdk version "21.0.12.1" 2026-08-18 LTS
3 OpenJDK Runtime Environment Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS)
4 OpenJDK 64-Bit Server VM Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS, mixed mode, sharing)
5
6 $ javac -version
```

```
7 javac 21.0.12.1
8
9 $ echo %JAVA_HOME%
10 C:\Program Files\Eclipse Adoptium\jdk-21.0.12.101-hotspot\
```



```
C:\WINDOWS\system32\cmd. x + v
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS, mixed mode, sharing)
javac 21.0.12.1
JAVA_HOME=C:\Program Files\Eclipse Adoptium\jdk-21.0.12.101-hotspot\
C:\Users\Samsung\Desktop\20271>
```

Figura 4: Verificación de `java -version` y `JAVA_HOME` en la terminal

Los tres comandos coinciden: la versión que reporta el ejecutable de Java es la misma que la del compilador, y `JAVA_HOME` apunta exactamente a la carpeta donde ese ejecutable vive. Esa coincidencia es, en la práctica, la prueba de que la instalación quedó bien encadenada de principio a fin.

3. Conclusión

La instalación del JDK es un proceso corto, pero con varios puntos donde suele fallar, casi siempre por descuido y no por dificultad técnica real. Los más comunes son:

- **Apuntar `JAVA_HOME` a la carpeta bin en vez de a la raíz del JDK.** Es el error más frecuente. Herramientas como Maven esperan que `JAVA_HOME` sea la carpeta que *contiene* a `bin`, no `bin` en sí misma; el síntoma típico es un error de «no se encuentra el ejecutable» aunque `java -version` funcione bien desde el PATH.
- **No abrir una terminal nueva después de configurar la variable.** Las variables de entorno se leen una sola vez, al arrancar el proceso. Una terminal o un IDE que ya estaban abiertos antes del cambio van a seguir sin ver `JAVA_HOME` hasta que se reinicien.
- **Instalar solo el JRE en vez del JDK.** El JRE (Java Runtime Environment) permite *ejecutar* programas Java pero no trae el compilador `javac`; para desarrollo hace falta específicamente el JDK.
- **Tener varias versiones del JDK instaladas y que el PATH apunte a una distinta de la que se acaba de configurar en `JAVA_HOME`.** Provoca que `java -version` y `javac -version` reporten números distintos, lo que suele romper la compilación de forma difícil de diagnosticar si no se revisan ambos comandos por separado.
- **Descargar el instalador de arquitectura equivocada** (32 bits en un sistema de 64 bits, o viceversa), lo que puede impedir la instalación o generar errores de compatibilidad más adelante con otras herramientas de 64 bits del entorno de desarrollo.

Verificar con dos comandos distintos (`java -version` y `javac -version`) en vez de uno solo, y confirmar el valor de `JAVA_HOME` de forma explícita en vez de asumirlo, es la forma más directa de detectar cualquiera de estos problemas antes de que se manifieste como un error confuso en Maven, Gradle o el IDE más adelante en el curso.

4. Referencias Bibliográficas

- [1] Oracle Corporation. *Java SE Development Kit Documentation*. 2024. URL: <https://docs.oracle.com/en/java/javase/21/> (visitado 11-09-2026).
- [2] Eclipse Adoptium. *Eclipse Temurin – Installation*. 2024. URL: <https://adoptium.net/installation/> (visitado 11-09-2026).
- [3] Oracle Corporation. *Setting the Path and Classpath – The Java Tutorials*. 2024. URL: <https://docs.oracle.com/javase/tutorial/essential/environment/paths.html> (visitado 11-09-2026).